Содержание

Введение 3

Цели и задачи 4

1 Алгоритмы вычисления НОД 6

1.1 Метод последовательного сокращения 6

1.2 Метод простых множителей 8

1.3 Алгоритм Евклида 9

1.4 Особенности нахождения НОД для отрицательных чисел 10

2 Алгоритмы вычисления НОК 11

2.1 Метод последовательного умножения 11

2.2 Метод простых множителей 12

2.3 Метод с использованием НОД 13

2.4 Особенности нахождения НОД для отрицательных чисел. 14

3 Графический интерфейс 15

3.1 Windows Forms 15

4 Разработка программы поиска НОД и НОК 17

5 Разработка графического интерфейса 19

5.1 «MyForm.cpp» 19

5.2 «MyForm.h» 20

6 Результат работы программы 30

6.1 Отображение графического интерфейса 30

6.2 Попытка ошибочного ввода 30

6.3 Ввод отрицательных чисел 31

6.4 Вывод результата 32

Заключение 33

Список использованных источников 34

Приложение №1. Код в файле «MyForm.cpp»**Ошибка! Закладка не определена.**

Приложение №2. Код в файле «MyForm.h»**Ошибка! Закладка не определена.**

Введение

В современном мире программирование является одной из ключевых навыков, способных решать широкий спектр задач различной сложности. Одной из таких задач является нахождение наибольшего общего делителя (НОД) и наименьшего общего кратного (НОК) двух или более чисел. Эти математические концепции являются фундаментальными математическими операциями, широко применяемыми в различных областях науки, техники и информатики. Они играют важную роль в решении задач криптографии, оптимизации, разработке алгоритмов и многих других областях.

Однако, вместе с ростом требований к программному обеспечению возникает необходимость в создании пользовательских интерфейсов, которые бы сделали использование программ более удобным и доступным для широкого круга пользователей. В этом контексте становится актуальной задача разработки программы поиска НОД и НОК с использованием графического интерфейса.

Цели и задачи

Цель работы состоит в разработке функционального и эффективного программного решения для решения задач нахождения НОД и НОК, а также изучение принципов работы и применения алгоритмов, лежащих в их основе.

Актуальность данной курсовой работы заключается в том, что в современном информационном обществе существует постоянная потребность в эффективных алгоритмах для решения различных задач. Нахождение наибольшего общего делителя и наименьшего общего кратного является фундаментальной математической операцией, широко применяемой в различных областях. Разработка программы для автоматизированного нахождения НОД и НОК не только облегчит рутинные вычисления, но и ускорит процесс решения задач, повышая эффективность работы как в образовательных, так и в научных и практических областях. Таким образом, данная работа имеет актуальное значение для современного программирования и информационных технологий.

Новизна данной курсовой работы – это подход к решению классической математической задачи, объединяющем в себе теоретические знания и практические навыки программирования.

Предметы исследования: особенности языка программирования C++ и его применимость в создании графических интерфейсов, математические методы нахождения НОД и НОК.

Задачи, которые ставятся перед курсовой работой по разработке программы поиска НОД и НОК с использованием графического интерфейса:

- Изучение алгоритмов нахождения НОД и НОК: Проведение обзора различных математических методов и алгоритмов для эффективного нахождения наибольшего общего делителя и наименьшего общего кратного чисел.

- Реализация программы: Написание кода программы, которая будет осуществлять поиск НОД и НОК по введенным пользователем числам.
- Проектирование графического интерфейса: Разработка дизайна и функциональности пользовательского интерфейса, который будет интуитивно понятен и удобен для использования.
- Тестирование программы: Проведение тестирования разработанной программы на различных наборах тестовых данных для проверки ее корректности, стабильности и эффективности.

Практическая значимость курсовой работы заключается в создании программного инструмента, который облегчит выполнение математических операций нахождения наибольшего общего делителя и наименьшего общего кратного. Эта программа будет полезна для различных областей, включая научные исследования, инженерные расчеты, образование и повседневные вычисления. Графический интерфейс делает программу более удобной и доступной для пользователей всех уровней навыков, обеспечивая интуитивно понятный способ взаимодействия. Таким образом, разработка программы поиска НОД и НОК обеспечивает ее практическую применимость в различных сферах деятельности и повседневной жизни, содействуя улучшению процесса решения математических задач.

1 Алгоритмы вычисления НОД

Для вычисления наибольшего общего делителя нескольких чисел существуют всего три метода:

- Метод последовательного сокращения.
- Метод простых множителей.
- Алгоритм Евклида.

Так же существуют некоторые особенности нахождения НОД у отрицательных чисел.

1.1 Метод последовательного сокращения

Метод последовательного сокращения, также известный как метод последовательного уменьшения наименьшего числа, используется для нахождения наибольшего общего делителя для нескольких чисел, последовательно уменьшая наименьшее число в наборе.

Описание метода:

- Инициализируется переменная d значением, равным минимальному числу из данного списка чисел.
- Проверяется, делится ли d на все числа из списка без остатка. Если делится, то текущее значение d является НОД. Алгоритм завершается.
- Если d не делится на все числа без остатка, d уменьшается на единицу. Повторяются шаги 2-3 до тех пор, пока не будет найден НОД или пока d не станет равным 1.

Применение метода последовательного сокращения к числам 12, 18 и 24:

- Наименьшее число в списке: 12. Инициализация d = 12.
- Проверка делимости 12 на все числа:

12 ÷ 12 = 1 (остаток 0),

18 ÷ 12 = 1 (остаток 6),

24 ÷ 12 = 2 (остаток 0).

d не делится на все числа

- Уменьшение d на 1: d = 11.
- Проверка делимости 11 на все числа:

  12 ÷ 11 = 1 (остаток 1),

  18 ÷ 11 = 1 (остаток 7),

  24 ÷ 11 = 2 (остаток 2).

  d не делится на все числа.

- Уменьшение d на 1: d = 10.
- Проверка делимости 10 на все числа:

  12 ÷ 10 = 1 (остаток 2),

  18 ÷ 10 = 1 (остаток 8),

  24 ÷ 10 = 2 (остаток 4).

  d не делится на все числа.

- Продолжение уменьшения d и проверка делимости продолжаются до d = 6:

  12 ÷ 6 = 2 (остаток 0),

  18 ÷ 6 = 3 (остаток 0),

  24 ÷ 6 = 4 (остаток 0).

  d делится на все числа.

- Следовательно, НОД для чисел 12, 18 и 24 равен 6.

Метод последовательного уменьшения хоть и прост в понимании и реализации, но не является самым оптимальным в большинстве случаев, особенно когда числа, для которых находится НОД, большие или имеют большую разницу между собой. Проблема этого метода заключается в том, что он может потребовать много итераций. Для более эффективного подхода лучше использовать алгоритмы, основанные на математических свойствах, такие как алгоритм Евклида или метод простых множителей.

1.2 Метод простых множителей

Метод простых множителей используется для нахождения наибольшего общего делителя двух или более чисел путем разложения числа на простые множители и перемножения между собой общих множителей для всех чисел [1].

– Разложение на простые множители:

  12: $2^2 \times 3^2$

  18: 2×3^2

  24: $2^3 \times 3$

– Нахождение общих простых множителей для всех трех чисел: 2 и 3.

– Умножение общих множителей:

  $2 \times 3 = 6$

– Следовательно, наибольший общий делитель для чисел 12, 18 и 24 равен 6.

Этот метод хорошо подходит для нахождения НОД для небольших чисел, особенно если они имеют относительно небольшое количество простых множителей. Однако для больших чисел, особенно если они имеют большое

количество простых множителей, этот метод может стать неэффективным из-за необходимости вычисления всех простых множителей каждого числа.

1.3 Алгоритм Евклида

Метод Евклида является одним из наиболее эффективных алгоритмов для нахождения наибольшего общего делителя двух чисел. Этот метод основан на принципе, что НОД двух чисел равен НОД одного из них и остатка от деления другого числа на первое. Для нахождения НОД для большего количества чисел метод Евклида применяется последовательно к парам чисел.

Описание метода Евклида для нахождения НОД для набора чисел:

- Начинается с двух чисел. Если чисел больше двух, выбирается любая пара для начала. Для двух чисел применяется стандартный алгоритм Евклида.
- Большее число делится на меньшее. Если результат деления не имеет остатка, то меньшее число и есть НОД. В противном случае большее число заменяется на остаток от деления [2]. Процесс повторяется до тех пор, пока не будет найдено число, которое делится нацело.
- После нахождения НОД для первой пары чисел, алгоритм применяется к НОД и следующему числу из набора. Процесс продолжается до тех пор, пока не останется одно число, которое и будет НОД для всего набора чисел.

Пример применения метода Евклида к числам 12, 18 и 24:

- Первые два числа: a = 12 и b = 18.
- Деление 18 на 12:

  18 ÷ 12 = 1 (остаток 6).

  Теперь a = 12, b = 6.

– Деление 12 на 6:

  12 ÷ 6 = 2 (остаток 0).

  Остаток 0, поэтому НОД равен последнему ненулевому делителю, то есть 6. НОД для чисел 12 и 18 равен 6.

– Теперь найденный НОД (6) используется для следующего числа (24):

  НОД (6, 24):

  24 ÷ 6 = 4 (остаток 0).

  НОД равен 6, поскольку деление нацело.

– Таким образом, наибольший общий делитель для чисел 12, 18 и 24 равен 6.

Этот метод эффективен и применим для любого количества чисел, поскольку основан на простой операции деления. Он позволяет найти НОД с минимальным количеством операций даже для очень больших чисел. Поэтому для оптимизации программ лучше использовать этот метод.

1.4 Особенности нахождения НОД для отрицательных чисел

Для нахождения наибольшего общего делителя отрицательных чисел можно воспользоваться модулями этих чисел.

Это связано с свойством чисел с противоположными знаками: числа n и -n имеют одинаковые делители [3].

2 Алгоритмы вычисления НОК

Для вычисления наименьшего общего кратного (НОК) нескольких чисел существуют всего три метода:

- Метод последовательного умножения.
- Метод простых множителей.
- Метод с использованием НОД.

Так же для вычисления НОК отрицательных чисел, как и при вычислении НОД существуют некоторые особенности

2.1 Метод последовательного умножения

Метод последовательного умножения — это один из методов нахождения НОК для нескольких чисел. Этот метод основан на простой идее последовательного умножения максимального числа из заданных на натуральные числа, пока результат не будет делиться на все числа из заданного списка.

Процесс метода:

- Находится наибольшее число из заданных.
- Это число умножается на последовательные натуральные числа (начиная с 1).
- Проверяется, делится ли полученное произведение на все числа из списка. Если делится, то текущее произведение становится новым значением НОК. Если не делится, продолжается умножение текущего значения на следующее натуральное число и проверяется снова.
- Процесс продолжается до нахождения НОК.

Пример с числами 12, 18 и 24:

– Наибольшее число из списка: 24.

– Умножение 24 на последовательные натуральные числа:

  $24 \times 1 = 24$ (не делится на 12 и 18)

  $24 \times 2 = 48$ (не делится на 12 и 18)

  $24 \times 3 = 72$ (делится на 12 и 18)

– Таким образом, НОК для чисел 12, 18 и 24 равен 72.

Однако этот метод не является наиболее эффективным для программного использования, поскольку требует множества итераций, особенно для больших чисел или чисел с большой разницей между ними. Для оптимизации программы предпочтительнее использовать более эффективные алгоритмы, например, алгоритм Евклида или метод простых множителей.

2.2 Метод простых множителей

Метод простых множителей — это математический метод, используемый для вычисления наименьшего общего кратного нескольких чисел путем разложения каждого числа на простые множители и выбора наибольших степеней этих множителей. Затем эти наибольшие степени простых множителей перемножаются, чтобы получить НОК.

Пример на трех числах 12, 18 и 24:

– Разложение на простые множители:

  12: $12 = 2^2 \times 3$

  18: $18 = 2 \times 3^2$

  24: $24 = 2^3 \times 3$

– Нахождение наибольших степеней простых множителей:

  Для 2: наибольшая степень - 3 (из 2^3).

Для 3: наибольшая степень - 2 (из 3^2).

– Вычисление НОК:

НОК = $2^3 \times 3^2 = 72$

Таким образом, НОК для чисел 12, 18 и 24 равен 72.

Метод простых множителей хотя и прост в понимании, может стать неэффективным при обработке больших чисел или большого количества чисел.

2.3 Метод с использованием НОД

Метод нахождения НОК с использованием НОД является одним из эффективных подходов к решению данной задачи. Этот метод основан на следующем свойстве: НОК двух чисел равен их произведению, деленному на их НОД. Используя это свойство, можно расширить алгоритм для нахождения НОД на случай нескольких чисел.

Описание метода:

– Находится НОД для первых двух чисел. Для этого используется любой известный алгоритм вычисления НОД, например, алгоритм Евклида.

– Вычисляется НОК для первых двух чисел с помощью формулы: НОК(a,b) $= \frac{a \times b}{\text{НОД}(a,b)}$, где a и b - числа, для которых был вычислен НОД на предыдущем шаге.

– Для каждого последующего числа из списка:

  Находится НОД текущего числа и ранее найденного НОК с помощью того же алгоритма НОД.

Вычисляется НОК для текущего числа и ранее найденного НОК с использованием формулы из пункта 2.

- Повторяются шаги 3 для всех оставшихся чисел в списке, пока НОК не будет вычислен для всех чисел.
- Полученное значение НОК после завершения всех вычислений является окончательным результатом.

Пример вычисления НОК для чисел 12, 18 и 24:

- Вычисление НОД для чисел 12 и 18 с помощью алгоритма Евклида: НОД (12,18) = 6.
- Вычисление НОК для 12 и 18 с использованием найденного НОД: НОК (12,18) $= \frac{12\times18}{6} = 36$.
- Вычисление НОД для 36 (ранее найденный НОК) и 24: НОД (36,24) = 12.
- Вычисление НОК для 36 и 24 с использованием найденного НОД: НОК (36,24) $= \frac{36\times24}{12} = 72$.
- Таким образом, НОК для чисел 12, 18 и 24 равен 72.

Этот метод эффективно использует свойство связи между НОК и НОД, позволяя вычислить НОК для нескольких чисел, минимизируя количество вычислений. Считая, что необходимо вычислять и НОД, и НОК этот метод прекрасно подойдет для оптимизирования программы.

2.4 Особенности нахождения НОД для отрицательных чисел.

Для того, чтобы найти НОК отрицательных чисел, нужно найти НОК модулей, так как определение гласит, что наименьшее общее кратное — это натуральное число, меньше нуля оно быть не может [4].

3 Графический интерфейс

Графический интерфейс представляет собой способ представления информации и управления программой с помощью графических элементов, таких как кнопки, поля ввода и т. д. Это значительно отличается от командного интерфейса, где пользователь должен вводить текстовые команды для выполнения операций. Графический интерфейс делает программы более доступными для широкого круга пользователей, не требуя от них специальных знаний или навыков в области информационных технологий.

В этом контексте разработка графического интерфейса играет важную роль в создании программы, обеспечивая эффективное и приятное взаимодействие пользователя с программой. Поэтому было принято решение создать графический интерфейс, и для этого используем Windows Forms в качестве основного инструмента разработки.

3.1 Windows Forms

Windows Forms — это платформа пользовательского интерфейса для создания классических приложений Windows. Она обеспечивает один из самых эффективных способов создания классических приложений с помощью визуального конструктора в Visual Studio. Такие функции, как размещение визуальных элементов управления путем перетаскивания, упрощают создание классических приложений.

В Windows Forms можно разрабатывать графически сложные приложения, которые просто развертывать, обновлять, и с которыми удобно работать как в автономном режиме, так и в сети [5].

В Windows Forms форма — это визуальная поверхность, на которой выводится информация для пользователя. Обычно приложение Windows Forms строится путем добавления элементов управления в формы и создания кода для реагирования на действия пользователя, такие как щелчки мыши или нажатия клавиш.

При выполнении пользователем какого-либо действия с формой или одним из ее элементов управления создается событие. Приложение реагирует на эти события с кодом и обрабатывает события при их возникновении.

В Windows Forms предусмотрено множество элементов управления, которые можно добавлять в формы. Например, элементы управления могут отображать текстовые поля, кнопки, раскрывающиеся списки, переключатели и даже веб-страницы.

4 Разработка программы поиска НОД и НОК

Программа содержит две функции для вычисления наибольшего общего делителя и наименьшего общего кратного нескольких чисел.

Функция «NODtwo» принимает два целых числа a и b и возвращает их НОД, используя алгоритм Евклида. Она продолжает делить большее число на меньшее до тех пор, пока остаток не станет равен нулю. На каждой итерации сохраняется значение b во временной переменной temp, а затем b обновляется как остаток от деления a на b, а a принимает значение temp. По завершении цикла возвращается значение a, которое и будет НОД двух входных чисел. Блок-схема данной функции представлена на рисунке 1.

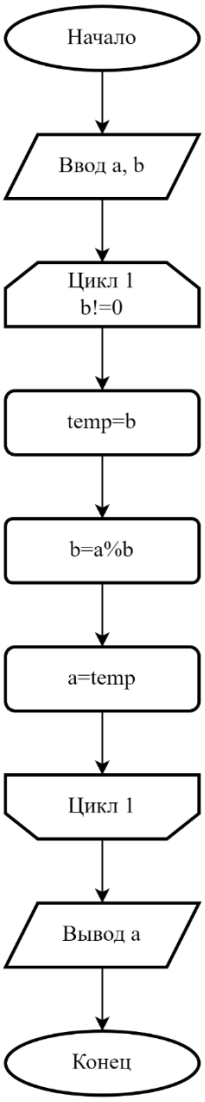

Рисунок 1 – Блок-схема функции «NODtwo»

Функция «NODandNOK» принимает массив целых чисел array, количество элементов в массиве n и флаг flag. Сначала она инициализирует переменные nod и nok значением абсолютной величины первого элемента массива array. Затем она проходит по остальным элементам массива и на каждой итерации обновляет nod, используя функцию «NODtwo», и nok, используя формулу НОК(a,b) $= \frac{a \times b}{\text{НОД}(a,b)}$. По завершении цикла функция возвращает либо значение НОД, либо НОК, в зависимости от значения флага. Блок-схема данной функции представлена на рисунке 2.

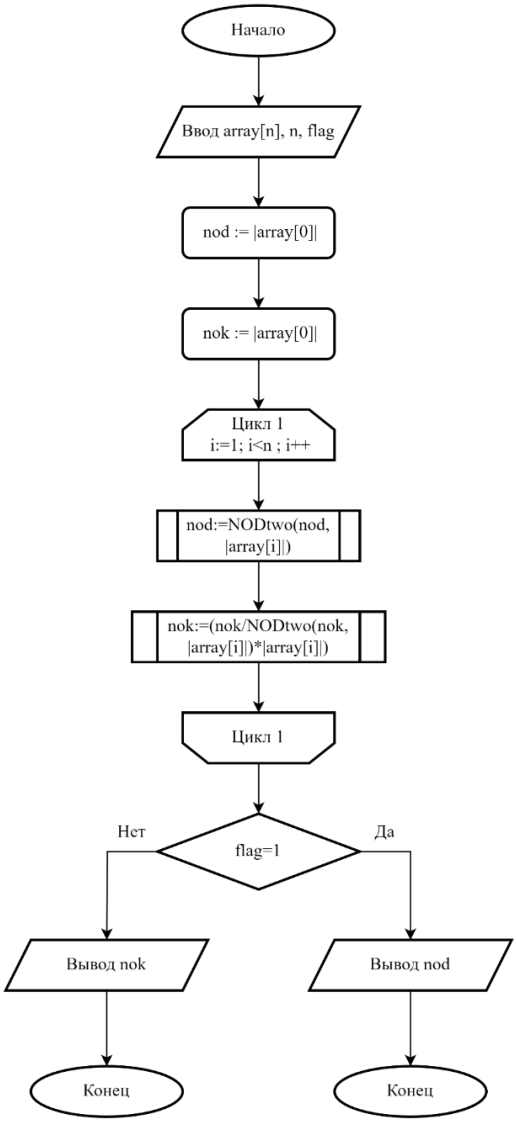

Рисунок 2 – Блок-схема функции «NODandNOK»

Данные функции будут вызваться в программе с графическим интерфейсом.

5 Разработка графического интерфейса

5.1 «MyForm.cpp»

Файл «MyForm.cpp» содержит функцию main(), которая инициализирует и запускает приложение Windows Forms. Код состоит из четырех частей. (рис. 3).

```
#include "MyForm.h"                                          1 часть

using namespace System;
using namespace System::Windows::Forms;                      2 часть

[STAThread]                                                  3 часть
void main(array<String^>^ arg) {

    Application::EnableVisualStyles();
    Application::SetCompatibleTextRenderingDefault(false);   4 часть

    КурсоваяНОДиНОК::MyForm form; //WinFormsTest - имя вашего проекта
    Application::Run(% form);
}
```

Рисунок 3 – Части кода в файле «MyForm.cpp»

В первой части кода подключается заголовочный файл «MyForm.h», который, содержит определение класса формы (MyForm), созданной в визуальном редакторе.

Во второй используется пространство имен System для доступа к классам и функциям из .NET Framework, а также пространство имен System::Windows::Forms для доступа к классам и функциям, связанным с созданием графического интерфейса на платформе Windows.

Третья часть - это атрибут, который указывает, что функция main() будет выполняться в однопоточном апартаменте. Это необходимо для создания приложений Windows Forms.

В четвертой части кода происходит инициализация стилей интерфейса и устанавливается рендеринг текста по умолчанию. Затем создается объект класса MyForm и запускается главный цикл приложения с помощью функции.

5.2 «MyForm.h»

Файл «MyForm.h» содержит объявления классов и функций, связанных с созданием и управлением формами в приложениях, разрабатываемых с использованием библиотеки или фреймворка для создания графического пользовательского интерфейса. В этом файле могут быть определены классы форм, их методы и свойства, а также обработчики событий, связанные с элементами управления на форме. Код в файле «MyForm.h» состоит из двенадцати частей (рис. 4, рис. 5, рис. 6, рис. 7, рис. 8, рис. 9, рис. 10).

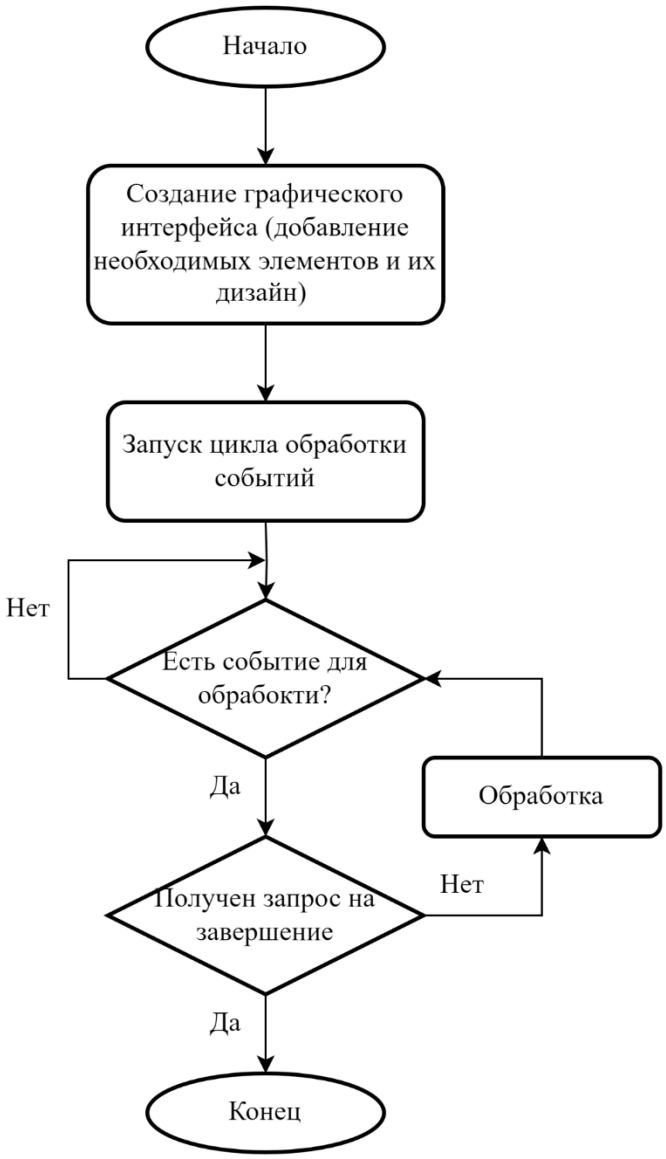

Рисунок 4 – Блок-схема работы графического интерфейса

Первая часть кода предварительно включает заголовочный файл <Windows.h>, который содержит определения и объявления для многих функций и структур, связанных с операционной системой Windows. Он предоставляет доступ к широкому спектру функций, таких как управление окнами, работа с файлами, вызов системных функций и т.д. В программе данный заголовочный файл нужен для работы с модулями переменных.

Вторая часть – это функции для вычисления наибольшего общего делителя и наименьшего общего кратного нескольких чисел.

Третья часть объявляет пространство имен с именем «КурсоваяНОДиНОК», которое вызывается в файле «MyForm.cpp».

Четвертая часть содержит директивы using namespace, которые позволяют использовать элементы из указанных пространств имен без явного указания префикса:

- System: Это основное пространство имен .NET Framework. Оно содержит базовые типы данных, классы и методы, необходимые для работы с операционной системой, файлами, потоками ввода-вывода и другими базовыми аспектами программирования.
- Пространство имен System::ComponentModel определяет целый ряд атрибутов (и других типов), позволяющих описать то, как должны вести себя элементы управления в режиме проектирования [6].
- Пространство имен System::Collections содержит классы и интерфейсы, которые служат для поддержки наборов (или коллекций) данных, организованных в виде стандартных структур данных – список, хэш-таблица и т. д. Кроме этого, некоторые интерфейсы из описываемого пространства имен удобны для использования при сортировках, перечислении элементов массивов и структур [7].
- System::Windows::Forms: Это пространство имен, которое содержит классы и методы для создания графического пользовательского

интерфейса с использованием Windows Forms. Здесь находятся элементы управления, события, методы для работы с формами и компонентами.

- System::Data: Это пространство имен, которое содержит классы для работы с данными и базами данных в .NET Framework.
- System::Drawing: Содержит классы для рисования графики и работы с изображениями в .NET Framework.
- System::IO: Это пространство имен, которое содержит классы и методы для работы с потоками ввода-вывода, файлами и каталогами в .NET Framework.

Пятая часть представляет шаблон формы в Windows Forms приложении, который инициализирует и освобождает компоненты формы:

- public ref class MyForm : public System::Windows::Forms::Form: Эта строка определяет класс MyForm, который является открытым (public) и управляемым (ref), а также наследует функциональность от класса System::Windows::Forms::Form. что означает, что MyForm представляет собой окно приложения Windows Forms.
- MyForm(void) { InitializeComponent(); }: Это конструктор класса MyForm. Конструктор вызывает метод InitializeComponent(), который инициализирует компоненты формы.
- protected:~MyForm() { if (components) delete components; }: Это деструктор класса MyForm. Деструктор освобождает память, выделенную для компонентов формы. Он вызывается при уничтожении объекта MyForm. Если components существует (то есть не равен nullptr), он удаляется с помощью оператора delete.

```
#include <Windows.h>
int NODtwo(int a, int b) {//алгоритм Евклида для двух значений
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

int NODandNOK(int* array, int n, int flag) {//вычисление НОД и НОК
    int nod = abs(array[0]);
    int nok = abs(array[0]);
    for (int i = 1; i < n; i++) {
        nod = NODtwo(nod, abs(array[i]));
        nok = (nok / NODtwo(nok, abs(array[i])) * abs(array[i]));
    }
    if (flag == 1) return nod;
    else return nok;
}

namespace КурсоваяНОДиНОК {

    //подключение пространств
    using namespace System;
    using namespace System::ComponentModel;
    using namespace System::Collections;
    using namespace System::Windows::Forms;
    using namespace System::Data;
    using namespace System::Drawing;
    using namespace System::IO;

    /// Сводка для MyForm
    public ref class MyForm : public System::Windows::Forms::Form
    {
    public:MyForm(void) {
        InitializeComponent();// Инициализация компонентов
    }
    protected:~MyForm() {
        if (components) delete components;// Освободить все используемые ресурсы
    }
```

1 часть

2 часть

3 часть

4 часть

5 часть

Рисунок 5 – Первые пять частей когда в файле «MyForm.h»

Шестая часть определяет приватные члены класса MyForm, которые будут использоваться в программе:

– flag: Эта переменная flag используется для отслеживания текущего состояния программы. В программе она используется для определения шага ввода данных пользователем.

– n: Эта переменная n хранит количество чисел, вводимых пользователем.

– array: Это динамически выделенный массив целых чисел размером n. Он используется для хранения введенных пользователем значений.

- isMouseDown: Эта переменная isMouseDown используется для отслеживания состояния кнопки мыши. Если кнопка мыши нажата, значение будет true, иначе false.
- xOffset, yOffset;: Эти переменные xOffset и yOffset используются для хранения смещения позиции мыши при перетаскивании формы.
- messflag: Эта переменная messflag используется для отслеживания того, было ли уже показано сообщение пользователю. В программе она используется для предотвращения повторного показа сообщения.

Седьмая часть определяет различные элементы управления (кнопки, метки, текстовые поля и т. д.), которые будут использоваться в пользовательском интерфейсе программы:

- label_nok: Метка для отображения значения наименьшего общего кратного.
- label_now: Метка для отображения исходного массива чисел.
- label_message: Метка для отображения сообщений пользователю. . В программе она используется для запросов на ввод данных или предупреждений.
- button_ready: Кнопка, которую пользователь нажимает для подтверждения ввода данных и выполнения операций.
- vvod: Текстовое поле, в которое пользователь вводит данные.
- label_nod: Метка для отображения значения наибольшего общего делителя.
- button_clear: Кнопка для очистки всех данных и начала нового ввода.
- pictureBox1: Пиктограмма или изображение, которое может использоваться для дополнительной визуализации или оформления интерфейса.

– button_exit: Кнопка для выхода из программы.

– panel1 и panel2: Панели, которые используются для визуализации.

Поле components является контейнером компонентов, который используется для управления ресурсами, созданными компонентами формы, components объявлено как System::ComponentModel::Container^, что означает, что это ссылка на объект типа Container, предоставляемый пространством имен System::ComponentModel. Этот контейнер обеспечивает базовую функциональность управления ресурсами компонентов формы.

```
//необходимые переменные для работы программы
private: int flag = 0;
private: int n = 0;
private: int* array = new int[n];
bool isMouseDown = false;
int xOffset, yOffset;
int messflag = 0;


//элементы
private: System::Windows::Forms::Label^ label_nok;
private: System::Windows::Forms::Label^ label_now;
private: System::Windows::Forms::Label^ label_message;
private: System::Windows::Forms::Button^ button_ready;
private: System::Windows::Forms::TextBox^ vvod;
private: System::Windows::Forms::Label^ label_nod;
private: System::Windows::Forms::Button^ button_clear;
private: System::Windows::Forms::PictureBox^ pictureBox1;
private: System::Windows::Forms::Button^ button_exit;
private: System::Windows::Forms::Panel^ panel1;
private: System::Windows::Forms::Panel^ panel2;


/// Обязательная переменная конструктора.
private:System::ComponentModel::Container^ components;
```

6 часть

7 часть

Рисунок 6 – Шестая и седьмая части кода в файле «MyForm.h»

Восьмая часть создает и настраивает визуальные элементы пользовательского интерфейса с помощью метода InitializeComponent().

```
//Дизайн
void InitializeComponent(void)
{
    System::ComponentModel::ComponentResourceManager^ resources = (gcnew System::ComponentModel::ComponentResourceManager(MyForm::typeid));
    this->label_nok = (gcnew System::Windows::Forms::Label());
    this->label_now = (gcnew System::Windows::Forms::Label());
    this->label_message = (gcnew System::Windows::Forms::Label());
    this->button_ready = (gcnew System::Windows::Forms::Button());
    this->vvod = (gcnew System::Windows::Forms::TextBox());
    this->label_nod = (gcnew System::Windows::Forms::Label());
    this->button_clear = (gcnew System::Windows::Forms::Button());
    this->pictureBox1 = (gcnew System::Windows::Forms::PictureBox());
    this->button_exit = (gcnew System::Windows::Forms::Button());
    this->panel1 = (gcnew System::Windows::Forms::Panel());
    this->panel2 = (gcnew System::Windows::Forms::Panel());
    (cli::safe_cast<System::ComponentModel::ISupportInitialize^>(this->pictureBox1))->BeginInit();
    this->SuspendLayout();
    //
    // label_nok
    //
    this->label_nok->Font = (gcnew System::Drawing::Font(L"Haettenschweiler", 21.85714F, System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
    this->label_nok->ForeColor = System::Drawing::Color::AntiqueWhite;
    this->label_nok->Location = System::Drawing::Point(388, 408);
    this->label_nok->Margin = System::Windows::Forms::Padding(2, 0, 2, 0);
    this->label_nok->Name = L"label_nok";
    this->label_nok->Size = System::Drawing::Size(650, 60);
    this->label_nok->TabIndex = 1;
    this->label_nok->Text = L"НОК: ";
    //
    // label_now
    //
    this->label_now->Font = (gcnew System::Drawing::Font(L"Haettenschweiler", 20.14286F, System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
    this->label_now->ForeColor = System::Drawing::Color::PowderBlue;
    this->label_now->Location = System::Drawing::Point(40, 238);
    this->label_now->Margin = System::Windows::Forms::Padding(2, 0, 2, 0);
    this->label_now->Name = L"label_now";
    this->label_now->Size = System::Drawing::Size(945, 76);
    this->label_now->TabIndex = 3;
    this->label_now->Text = L"Изначальный массив с числами: {}";
    //
    // label_message
    //
    this->label_message->Font = (gcnew System::Drawing::Font(L"Haettenschweiler", 24, System::Drawing::FontStyle::Italic, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(204)));
    this->label_message->ForeColor = System::Drawing::Color::LightSteelBlue;
    this->label_message->Location = System::Drawing::Point(26, 71);
    this->label_message->Margin = System::Windows::Forms::Padding(2, 0, 2, 0);
    this->label_message->Name = L"label_message";
    this->label_message->Size = System::Drawing::Size(1012, 65);
    this->label_message->TabIndex = 4;
    this->label_message->Text = L"Введите колличество чисел";
```

8 часть

Рисунок 7 – Пример восьмой части кода в файле «MyForm.h»

Девятая часть – это функции, которые представляют обработчики событий для различных элементов управления на форме. Краткое описание каждой из них:

- vvod_TextChanged: Этот метод вызывается каждый раз, когда текст в текстовом поле vvod изменяется. Обычно используется для реагирования на изменения вводимого пользователем текста.
- button_Clear: Этот метод вызывается при нажатии на кнопку "Очистить". Он просто вызывает функцию Clear(), которая, отвечает за очистку формы.
- button1_Click: Этот метод вызывается при нажатии на кнопку "x". Он закрывает текущую форму, вызывая метод Close().
- button1_Click_1: Этот метод вызывается при нажатии на кнопку "Готово", вызывая метод Base().

– vvod_KeyUp: Этот метод вызывается каждый раз, когда пользователь отпускает клавишу на клавиатуре, находясь в текстовом поле vvod. В данном случае он проверяет, была ли отпущена клавиша Enter, и если да, то вызывает метод Base(). Данный метод был создан для удобства ввода данных пользователем.

Десятая часть – это метод Clear(), который предназначен для очистки всех данных на форме и возвращения ее в исходное состояние.

```
//pragma endregion
private: System::Void vvod_TextChanged(System::Object^ sender, System::EventArgs^ e) {
}
private: System::Void button_Clear(System::Object^ sender, System::EventArgs^ e) {
    Clear();
}
private: System::Void button1_Click(System::Object^ sender, System::EventArgs^ e) {
    this->Close();
}
private: System::Void button1_Click_1(System::Object^ sender, System::EventArgs^ e) {
    Base();
}
private: System::Void vvod_KeyUp(System::Object^ sender, System::Windows::Forms::KeyEventArgs^ e) {
    if (e->KeyCode == Keys::Enter) {
        Base();
    };
}

//функции с элементами
private: System::Void Clear() {//очистка формы
    this->label_now->Text = L"Изначальный массив с числами: {}";
    this->label_message->Text = L"Введите колличество чисел";
    this->label_nod->Text = L"НОД: ";
    this->label_nok->Text = L"НОК: ";
    vvod->Text = "";
    flag = 0;
    messflag = 0;
}
```

9 часть

10 часть

Рисунок 8 – Девятая и десятая части кода в файле «MyForm.h»

Одиннадцатая часть – это метод, Base(), представляет собой основную логику программы. Краткое описание его функциональности:

– Увеличение счетчика flag на единицу.

– Если flag равен 1, значит, это первый раз, когда метод вызывается. В этом случае выполняется следующее: пытается преобразовать текст из текстового поля vvod в целое число и сохраняет его в переменную n. Проверяет, чтобы введенное число было больше 0. Если это не так, вызывается функция Clear(), и пользователю выводится сообщение об ошибке.

– Иначе пользователю выводится сообщение о том, что нужно ввести первое число.

– Если flag больше 1 и меньше или равен n + 1, значит, пользователь уже начал вводить числа. В этом случае: В текстовое поле label_message выводится сообщение о том, какое число нужно ввести сейчас. Пытается преобразовать введенный текст в целое число и сохраняет его в соответствующий элемент массива array. Если введенное число отрицательное и это первый раз, когда это происходит (messflag равен 0), выводится сообщение об этом. Обновляется текст в элементе label_now, отображающем текущий массив чисел.

– Если flag равен n + 1, это значит, что все числа введены, и можно вывести результат: В элемент label_nod добавляется текст, представляющий результат вычисления НОД. В элемент label_nok добавляется текст, представляющий результат вычисления НОК. Пользователю выводится сообщение о завершении ввода.

– Очищается текстовое поле vvod.

```
private: System::Void Base() {
    flag += 1;
    if (flag == 1) {
        try {//Ввод размера масива и проверка правильности ввода
            n = Convert::ToInt32(vvod->Text);
            if (n <= 0) {
                Clear();
                this->label_message->Text = L"Неверный ввод! Введите колличество чисел заново!";
            }
            else this->label_message->Text = L"Введите число номер 1";
        }
        catch (FormatException^) {
            Clear();
            this->label_message->Text = L"Неверный ввод! Введите колличество чисел заново!";
        }

    }
    else {
        if (flag <= n + 1) {//Заполнение массива и проверка правильности ввода
            this->label_message->Text = L"Введите число номер " + (flag);
            try {
                array[flag - 2] = Convert::ToInt32(vvod->Text);
                if (array[flag - 2] < 0 && messflag == 0) {
                    messflag = 1;
                    MessageBox::Show(L"НОК - это натуральное число, меньше нуля оно быть не может. Поэтому НОК берется от модулей чисел", L"Обрати Внимание!", MessageBoxButtons::OK, MessageBoxIcon::Information);
                }
            }
            catch (FormatException^) {
                Clear();
                this->label_message->Text = L"Неверный ввод! Введите колличество чисел заново!";
            }
            this->label_now->Text = L"Изначальный массив с числами:{ ";
            for (int i = 0; i < flag - 1; i++) {
                this->label_now->Text += array[i] + L" ";
            }
            this->label_now->Text += L"}";
        }

    }
    if (flag == n + 1) {// вывод результата
        this->label_nod->Text += NODandNOK(array, n,1);
        this->label_message->Text = L"Ваш результат!";
        this->label_nok->Text += NODandNOK(array, n,0);
    }
    vvod->Text = "";
}
```

11 часть

Рисунок 9 – Одиннадцатая часть кода в файле «MyForm.h»

Двенадцатая часть программы отвечает за перемещение формы при щелчке мыши и ее перетаскивании. Принцип работы данной части:

- MyForm_MouseDown: Этот метод вызывается при нажатии кнопки мыши на форме. Он проверяет, была ли нажата левая кнопка мыши (e->Button == Windows::Forms::MouseButtons::Left). Если это так, то устанавливает флаг isMouseDown в значение true, что указывает на то, что кнопка мыши нажата.
- MyForm_MouseMove: Этот метод вызывается, когда мышь перемещается по форме. Если флаг isMouseDown установлен в true, это означает, что кнопка мыши все еще нажата, и форма должна переместиться. Метод обновляет положение формы в соответствии с текущим положением мыши.
- MyForm_MouseUp: Этот метод вызывается, когда кнопка мыши отпускается. Если отпущена левая кнопка мыши (e->Button == Windows::Forms::MouseButtons::Left), флаг isMouseDown устанавливается в false, что указывает на то, что кнопка мыши больше не нажата, и перемещение формы должно прекратиться.

```
    //перемещение формы
    private: Point mouseOffset;
    private: System::Void MyForm_MouseDown(System::Object^ sender, System::Windows::Forms::MouseEventArgs^ e) {
        if (e->Button == Windows::Forms::MouseButtons::Left)
        {
            if (e->Button == Windows::Forms::MouseButtons::Left)
            {
                xOffset = -e->X - SystemInformation::FrameBorderSize.Width;
                yOffset = -e->Y - SystemInformation::FrameBorderSize.Height;
                mouseOffset = System::Drawing::Point(xOffset, yOffset);
                isMouseDown = true;
            }
        }
    }
    private: System::Void MyForm_MouseMove(System::Object^ sender, System::Windows::Forms::MouseEventArgs^ e) {
        if (isMouseDown)
        {
            Point mousePos = Control::MousePosition;
            mousePos.Offset(mouseOffset.X, mouseOffset.Y);
            Location = mousePos;
        }
    }
    private: System::Void MyForm_MouseUp(System::Object^ sender, System::Windows::Forms::MouseEventArgs^ e) {
        if (e->Button == Windows::Forms::MouseButtons::Left)
        {
            isMouseDown = false;
        }
    }
};
}
```

12 часть

Рисунок 10 – Двенадцатая часть кода в файле «MyForm.h»

6 Результат работы программы

6.1 Отображение графического интерфейса

При запуске программы она успешно отобразила графический интерфейс (рис. 11).

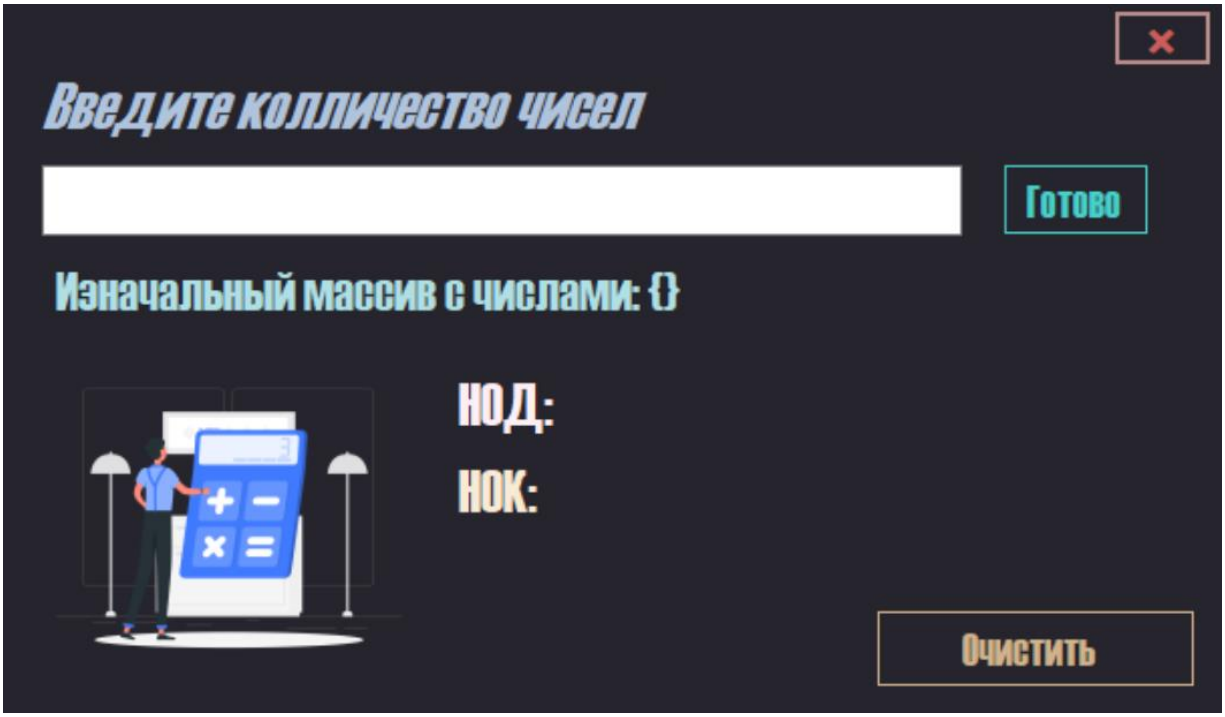

Рисунок 11 – Отображение графического интерфейса при запуске

6.2 Попытка ошибочного ввода

При ошибочном вводе пользователя, программа вывела сообщение об ошибке и запросила ввод корректного действия еще раз (рис. 12).

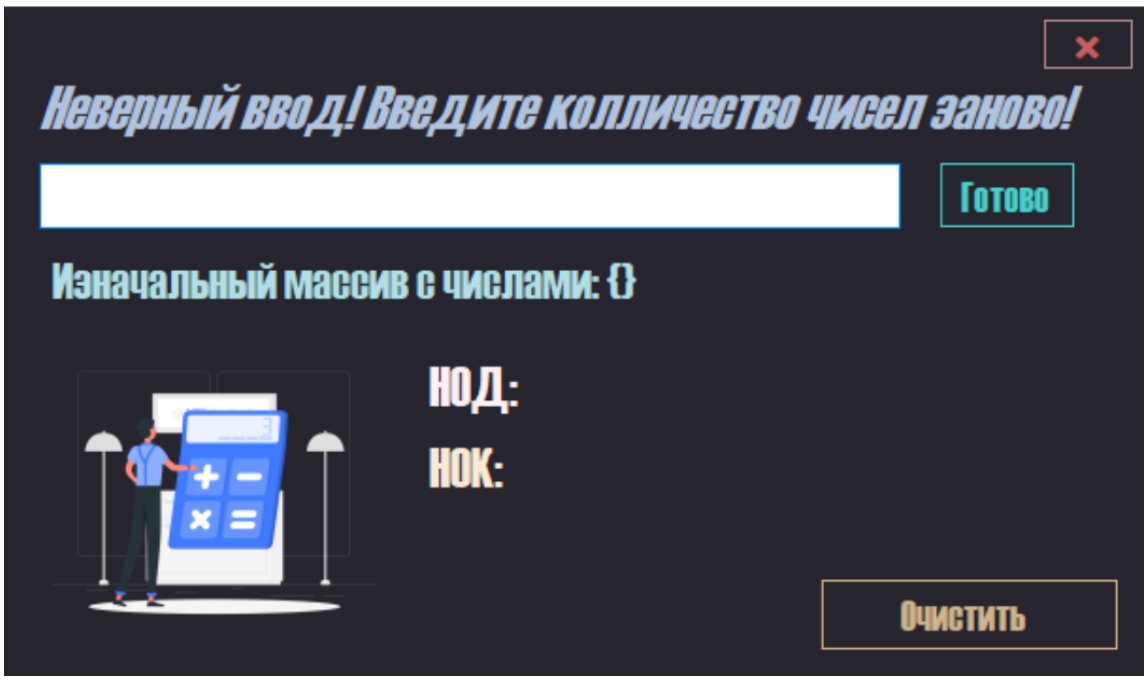

Рисунок 12 – Отображение графического интерфейса при вводе неверных значений

6.3 Ввод отрицательных чисел

При вводе отрицательных чисел программа вывела предупреждение (рис. 13).

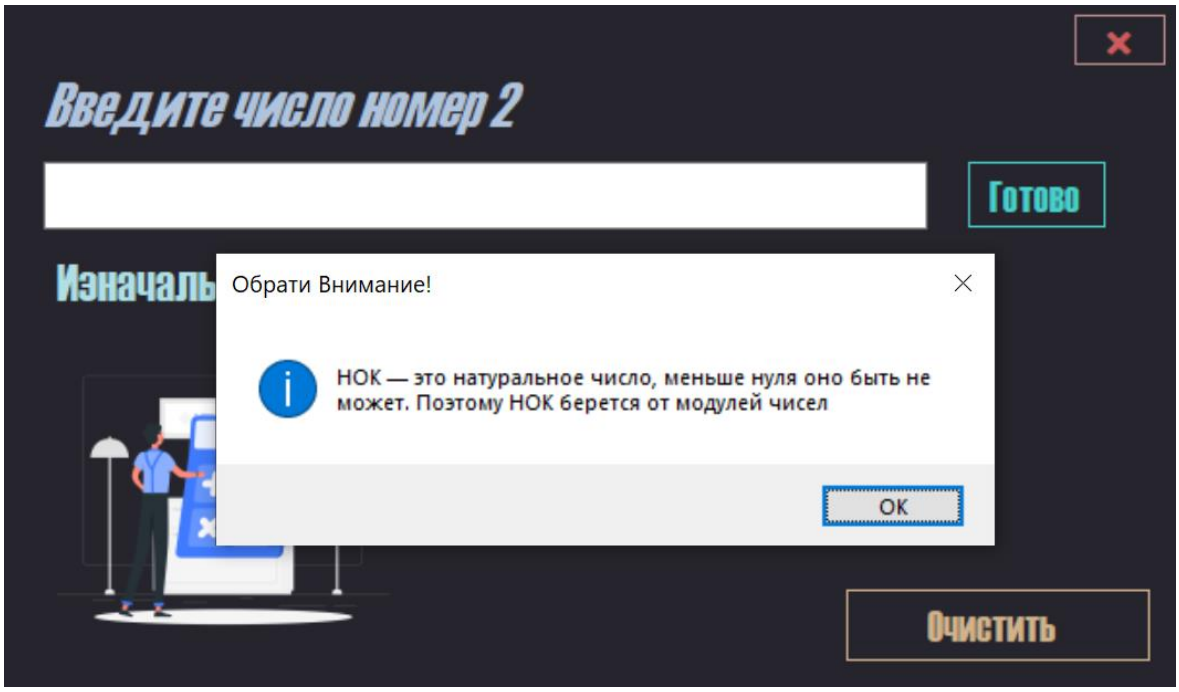

Рисунок 13 – Отображение графического интерфейса при вводе отрицательных значений

6.4 Вывод результата

При корректных данных, программа успешно вычислила значения НОД и НОК (рис. 14).

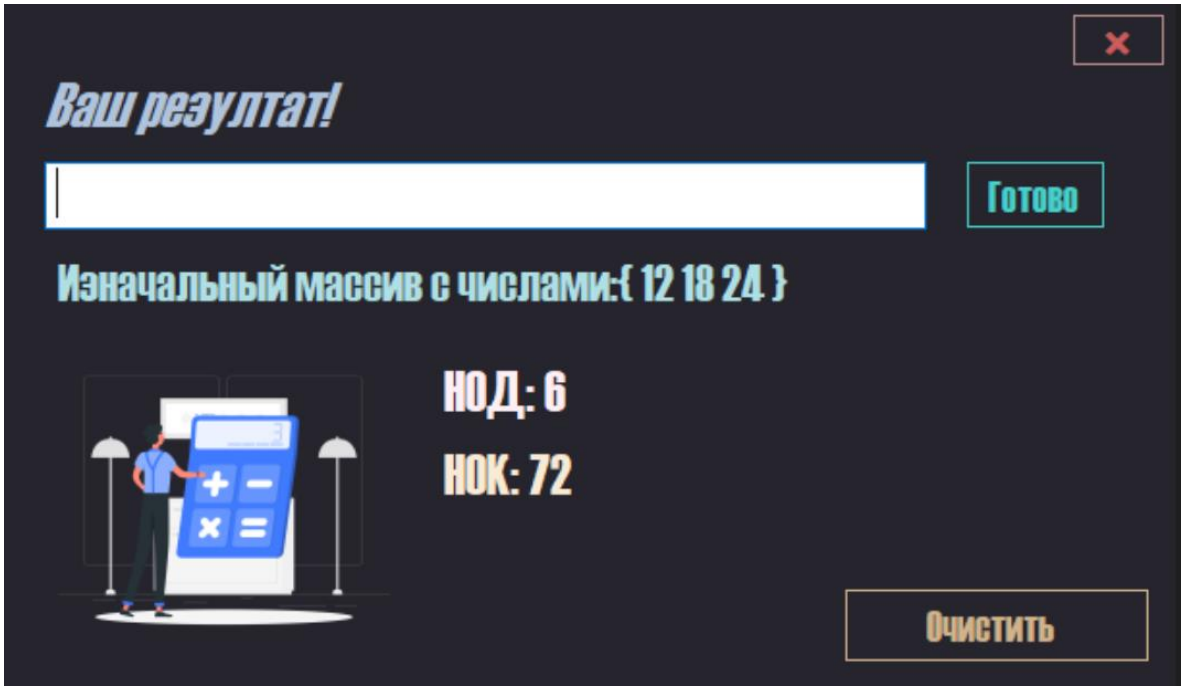

Рисунок 14 – Отображение графического интерфейса при выводе результатов вычисления НОД и НОК нескольких чисел

Заключение

В ходе курсовой работы была разработана программа для поиска наибольшего общего делителя и наименьшего общего кратного двух или более чисел. Реализация программы включала в себя изучение и применение различных алгоритмов, таких как алгоритм Евклида, метод простых множителей, метод последовательного сокращения, метод с использованием НОД и метод последовательного умножения, а также создание удобного пользовательского интерфейса.

Основная цель работы состояла в создании функциональной и эффективной программы, способной оперативно находить НОД и НОК для различных наборов чисел. Для достижения этой цели были применены современные методы программирования и тестирования, что позволило создать надежное программное обеспечение.

В процессе выполнения курсовой работы был проведен анализ алгоритмов, выбраны наиболее подходящие для реализации в задаче поиска НОД и НОК, а также разработан и протестирован пользовательский интерфейс программы. Полученные результаты демонстрируют корректную работу программы и ее соответствие заявленным требованиям.

В заключение можно отметить, что выполнение данной курсовой работы позволило углубить знания по алгоритмам и структурам данных, а также приобрести опыт разработки программного обеспечения с использованием современных технологий. Полученные навыки и знания будут полезными в дальнейшей профессиональной деятельности в области информатики и программирования.

Список использованных источников

1 Наибольший общий делитель (НОД), свойства и формулы / Skysmart [Электронный ресурс]. URL: https://skysmart.ru/articles/mathematic/naibolshij-obshchij-delitel (Дата обращения: 01.05.2024).

2 Алгоритм Евклида - нахождение наибольшего общего делителя / Лаборатория линуксоида [Электронный ресурс]. URL: https://younglinux.info/algorithm/euclidean (Дата обращения: 01.05.2024)

3 Мальцевская И., Нахождение НОД по алгоритму Евклида и с помощью разложения на простые множители [Электронный ресурс]. URL: https://zaochnik.com/spravochnik/matematika/delimost/nahozhdenie-nod/ (Дата обращения: 02.05.2024)

4 Определение НОК, способы его нахождения и характеристики [Электронный ресурс]. URL: https://dzodzo.ru/mathssub/naimenshee-obshhee-kratnoe/ (Дата обращения: 02.05.2024)

5 Руководство по классическим приложениям (Windows Forms .NET) / Microsoft Learn [Электронный ресурс]. URL: https://learn.microsoft.com/ru-ru/dotnet/desktop/winforms/overview/?view=netdesktop-8.0 (Дата обращения: 05.05.2024)

6 Пространство имен System.ComponentModel / StudFiles [Электронный ресурс]. URL: https://studfile.net/preview/9455892/page:368/ (Дата обращения: 06.05.2024).

7 Пространство имен system.Collections/ StudFiles [Электронный ресурс]. URL: https://studfile.net/preview/9478328/page:26/#:~:text=Пространство%20имен%20System.Collections%20содержит%20классы,перечислении%20элементов%20массивов%20и%20структур (Дата обращения: 06.05.2024).